AgentGS: Empowering Earth Science Agents with Plan Composition and Completion over Skill Hypergraphs

Anonymous Author(s)
Submission Id: TBD

Abstract

Geoscientific analyses require multiple tools to exchange data and intermediate results. Planning around individual tools ties workflows to specific implementations, while reusing entire strategies limits the combination of local operations across tasks. We introduce AgentGS, an agent that composes analytical workflows from reusable operations before selecting their implementations. A hypergraph organizes strategy knowledge, allowing the agent to combine operations across strategies while preserving their prerequisites and usage constraints. When the initial workflow is incomplete, AgentGS searches for compatible operations to connect its fragments. The resulting workflow guides tool selection, intermediate-result propagation, and recovery from execution failures. We also introduce GeoToolEval, a benchmark spanning diverse geoscientific research themes that evaluates final answers alongside tool workflows. Experiments show that AgentGS outperforms existing methods in answer quality and tool-workflow agreement, demonstrating the effectiveness of structured composition and completion for geoscientific tool orchestration.

CCS Concepts

• **Computing methodologies** → **Intelligent agents**; *Planning and scheduling*.

Keywords

LLM Agents, Compositional Planning, Tool Orchestration, Skill Hypergraph, Scientific Agent

ACM Reference Format:
Anonymous Author(s). 2027. AgentGS: Empowering Earth Science Agents with Plan Composition and Completion over Skill Hypergraphs. In *Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027), Hanoi, Vietnam, 3–7 May 2027*, IFAAMAS, 17 pages.

1 Introduction

Earth science increasingly combines observational data, computational models, and methods from multiple disciplines [15]. Analyses of climate, resources, and disasters connect data retrieval, alignment, variable derivation, modeling, and verification. Scientific agents built around large language models (LLMs) can help organize these stages into analytical workflows [4, 5]. Reliable execution requires each operation to receive the appropriate upstream result. Comparing a composite raster's mean with a reference value, for example, requires a statistical operation between raster compositing and scalar comparison. A missing intermediate operation leaves the workflow incomplete even when the selected tools are individually relevant. Orchestration must therefore account for both the operations required by a task and the data exchanged between them.

GeoAgent and Earth-Agent demonstrate multistep geoscientific tool use through planning and execution [1, 5]. Planning directly over tools, however, couples workflows to concrete interfaces, while reusing entire strategies limits the extraction of local operations for new tasks. An extracted operation must also retain the prerequisites and implementation constraints imposed by its source strategy. For example, two strategies may share a spatial operation while requiring different preprocessing or input objects. Reusing the operation's name alone does not preserve these requirements. The challenge is to combine *operation-level reuse* with *constraint preservation*, composing operations across strategies without losing the conditions needed for their execution.

AgentGS addresses this challenge through plan composition and completion over skill hypergraphs (Figure 1). Its Skill–Capability–Tool representation separates strategies, reusable operations, and concrete implementations. Shared operations connect strategies, while their source-specific dependencies constrain reuse. AgentGS expands selected operations with mandatory predecessors and searches other strategies for compatible branches that connect incomplete fragments. Candidate connections must satisfy prerequisite, data-compatibility, and tool-availability constraints. In the raster example, completion supplies the missing statistical operation and binds its output to the comparison. The system then selects tools, propagates intermediate results, and supports local recovery from execution failures. This separates the organization of an analytical workflow from the choice of its concrete implementations.

Evaluating orchestration also requires examining how answers are obtained. Recent benchmarks assess multistep geoscientific tool use [5, 6, 13]. We introduce GeoToolEval to pair answer evaluation with reference-workflow comparison across research themes. Tool-call annotations support analysis of selection, arguments, order, and execution efficiency within the same task. Single-step and multistep records distinguish basic tool use from workflows involving intermediate results, while execution traces support investigation of missing operations and failed calls. Our contributions are:

- AgentGS composes and completes workflows across strategies while preserving operation prerequisites and tool constraints before selecting concrete implementations.
- GeoToolEval provides 603 evaluation records across ten research themes, pairing reference answers with annotated tool workflows to assess outcomes and execution behavior.
- Experiments show that AgentGS outperforms the evaluated methods in answer quality and tool-workflow agreement.



This work is licensed under a Creative Commons Attribution International 4.0 License.

Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027), M. Baldoni, F. Fang, W. Yeoh, N. Yorke-Smith (eds.), 3–7 May 2027, Hanoi, Vietnam. © 2027 International Foundation for Autonomous Agents and Multiagent Systems (www.ifaamas.org).

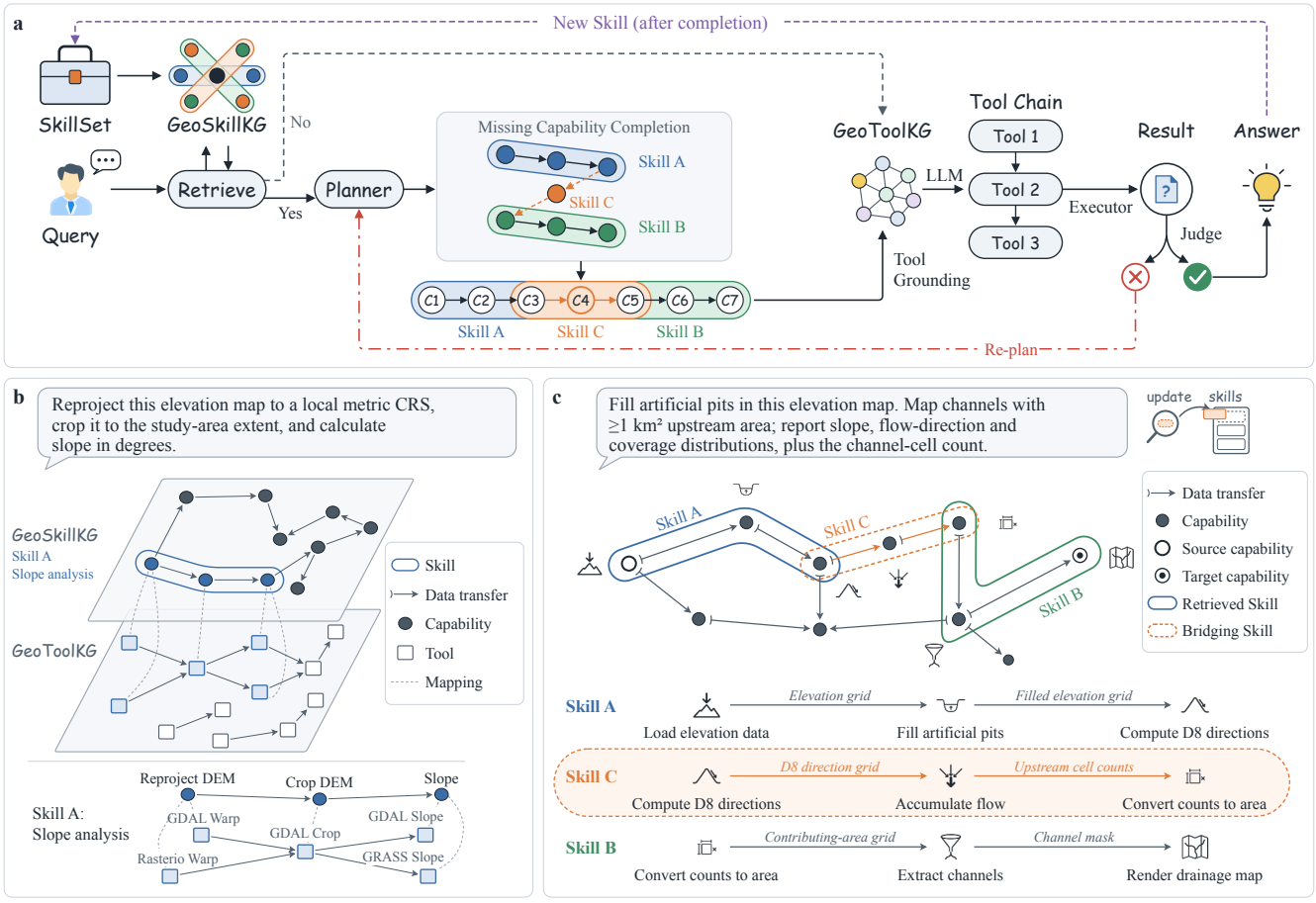


Figure 1: AgentGS hierarchical representation and overall workflow. (a) Retrieval, planning, completion, execution, verification, and Skill writeback. (b) Shared capabilities, strategy-specific dependencies, and tool implementations. (c) A connecting branch retrieved from another Skill. Verified compositions can be saved as candidate Skills.

2 Related Work

Planning and dependency representations. ReAct interleaves reasoning, actions, and observations [12]. ToolPlanner methods organize large tool collections [8, 10], while LLM+P and Generalised BDI Planning introduce explicit planning and goal management [7, 9]. GeoAgent and Earth-Agent apply multistep planning to geoscientific analysis [1, 5]. These methods establish planning and execution as central components of scientific agents.

Structured dependency representations guide composition. SciToolAgent organizes tools through a knowledge graph [4]; HyperAgent constructs a tool hypergraph from input/output schemas and searches for supporting tools [14]. AgentGS organizes shared operations and strategy-specific dependencies separately from tool implementations. Reused operations retain their source prerequisites and permitted tools, so fragments can be combined without adopting an entire strategy. Completion then searches declared branches to connect those fragments. The focus is on preserving strategy knowledge during local reuse, with concrete tool selection following workflow composition.

Geoscientific evaluation. GeoBench and EarthSE assess knowledge and scientific exploration without requiring tool execution [2, 11]. GeoBenchX evaluates multistage geospatial tasks against reference solutions [6]; Earth-Bench distinguishes autonomous planning from instruction following and records tool inputs and outputs [5]; GeoAgentBench checks invocation order, parameters, and map artifacts in a GIS sandbox [13]; and GeoNatureAgent Benchmark tests multitool analysis and recovery with real APIs [3].

GeoToolEval complements these benchmarks with coverage across ten research themes and scores for answers and tool workflows under common interfaces. This supports comparison across data types and workflow lengths. A reference workflow specifies one implementation, so agreement with its calls is assessed separately from answer correctness. Successful tool calls alone do not establish that downstream operations consumed the intended intermediate results. The annotations and execution records link calls to their inputs and outputs, supporting diagnosis of missing operations and execution errors, including incorrect bindings between otherwise appropriate tools.

3 AgentGS

AgentGS extends a tool-only agent with hierarchical strategy knowledge, Skill-guided plan composition, and automatic completion (Figures 1 and 2). Composition provides a task-specific starting plan; completion recovers missing connections before the plan is mapped to executable tools.

3.1 Hierarchical Organization of Skills and Tools

Individual tools provide little guidance about their joint use, while complete Skills may contain operations that a new task does not need. AgentGS exposes the internal operations of reusable Skills, combining strategy-level guidance with the flexibility to compose local operations across strategies.

Each Skill specifies an objective, typed inputs and outputs, and dependencies between operations. A shared operation, termed a *Capability*, maps to one or more implementing tools. We represent this organization as a hypergraph with operations as nodes and Skills as hyperedges. Shared operations connect strategies, while declared dependencies govern their composition.

Reuse preserves the prerequisites and tool constraints of the source Skill. Two strategies can therefore share an operation while requiring different preprocessing steps. Operations on different input objects remain separate, retaining the identity of the result consumed downstream. Appendix A gives the formal representation.

3.2 Skill-guided Plan Composition

Given a question and input resources, AgentGS retrieves Skills using the requested operations, domain, and available data types. The model selects goal operations from the retrieved candidates. The framework expands their mandatory predecessors and combines the selected fragments into an initial plan, preserving dependencies and permitted tools. Multiple required inputs can produce a branching plan.

This procedure heuristically prunes the planning space. Task relevance narrows the candidate strategies, and goal selection excludes unrelated optional operations while retaining required predecessors. Typed inputs and outputs guide connections between fragments. A raster-producing operation, for example, cannot directly supply a comparison that expects a scalar; the unresolved input identifies a gap for completion.

Each operation retains its source Skill, required inputs, and permitted implementations. These records constrain tool selection and preserve the operation's role in the workflow.

The plan distinguishes data dependencies from control dependencies. Data dependencies identify which earlier result an operation consumes; control dependencies specify ordering without making the preceding output an input. This distinction is retained when fragments from different Skills are composed.

The model thus receives a structured starting plan and scoped tool candidates. It can reuse part of a Skill without regenerating its declared prerequisites or adopting the whole workflow. When no Skill applies, the agent falls back to the existing tool-only planning path.

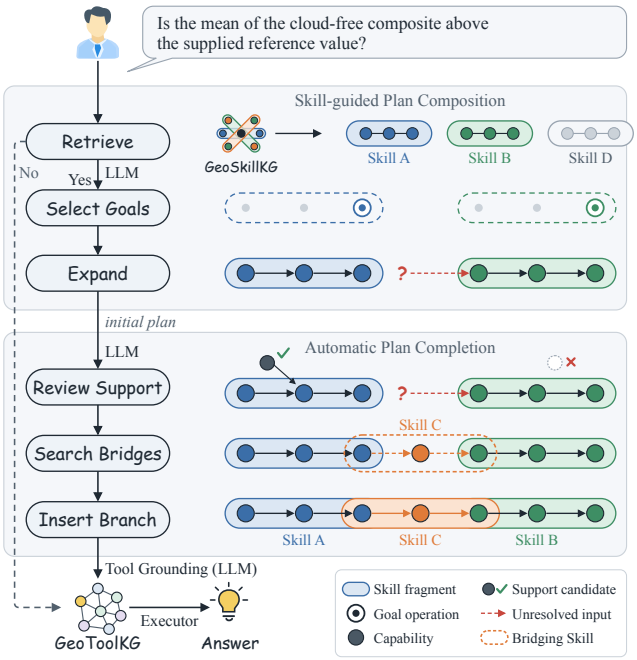


Figure 2: AgentGS planning pipeline. Composition turns retrieved Skills into an initial plan; completion inserts a bridging branch from another Skill to resolve an open input. Without an applicable Skill, planning falls back to the tool-only path.

3.3 Automatic Plan Completion

AgentGS examines the initial plan for omitted operations and disconnected fragments. It proposes supporting operations from compatible Skills for task-relevance review, then searches other Skills for declared branches connecting active fragments. The search fixes the active boundaries and uses shared operations and typed interfaces to identify candidate connections within individual provider Skills.

A candidate must add a missing operation, satisfy its prerequisites, and have available tools within the declared scope. Insertion must preserve acyclic dependencies. A unique valid branch is added automatically; ambiguous alternatives are reported. This procedure is designed to repair omissions without requiring the model to generate every intermediate step.

Insertion preserves the entry operation's constraints and updates the receiving operation's dependencies and bindings. Recomposition verifies mandatory prerequisites; a branch requiring additional operations outside the accepted connection is rejected.

Figure 2 illustrates a connection between raster compositing and scalar comparison. Completion inserts a valid-pixel mean operation and binds the comparison to its output. Grounding selects concrete tools and preserves these producer–consumer bindings; an inconsistent grounding is rejected. The resulting plan uses the existing execution and answer-generation services. Search rules, execution details, and the complete example are provided in Appendices A, B, and E.

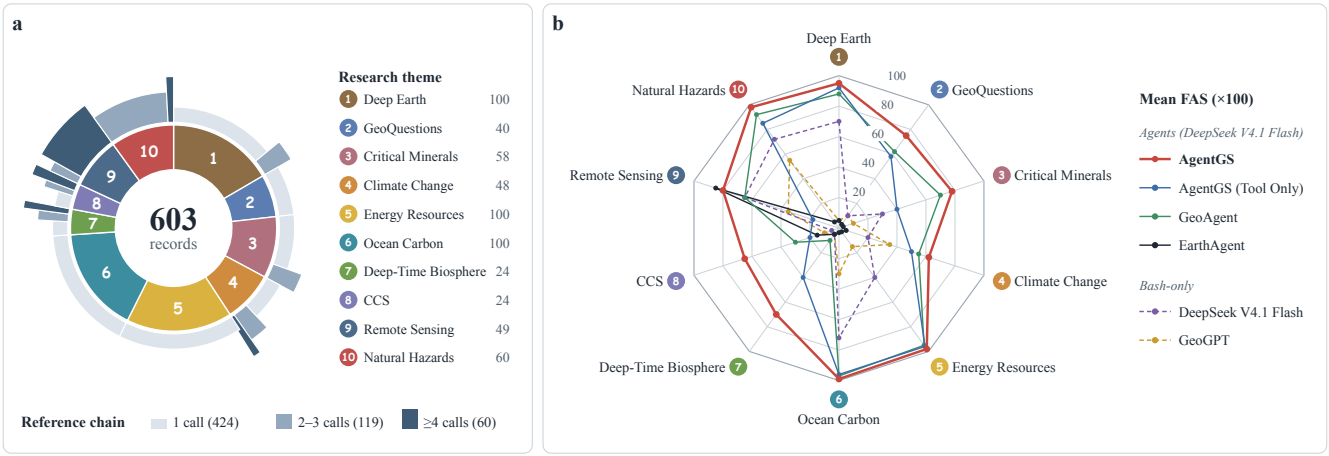


Figure 3: GeoToolEval composition and domain-specific performance. (a) The 603 evaluation records by research theme (inner ring); outer teeth split each theme by reference-chain length, with longer teeth for longer chains. (b) Mean FAS on each theme. The agent frameworks AgentGS, AgentGS (Tool Only), GeoAgent, and EarthAgent are all evaluated with DeepSeek V4.1 Flash (solid); DeepSeek V4.1 Flash and GeoGPT run as Bash-only agents (dashed).

4 GeoToolEval

We introduce GeoToolEval by integrating tasks from multiple geoscientific datasets and annotating their reference tool calls under a common set of interfaces. Each task pairs a scientific question with input resources, a reference answer, and a reference workflow, supporting evaluation of both the result and the tool use required to obtain it.

4.1 Dataset Construction and Coverage

GeoToolEval combines existing question-answer tasks with tasks derived from scientific data. The former include selected questions from GeoQuestions1089, Earth-Bench [5], and habitable_earth_ dataset. We retain their questions and reference answers while mapping the required operations to the registered tool interfaces. For data-derived tasks, questions specify identifiable objects and computational relationships, and reference answers are obtained through the corresponding analysis.

The dataset contains 603 evaluation records across ten research themes, covering resources such as Earth-structure models, mineral inventories, climate observations, remote-sensing imagery, and disaster-event records (Table 6; Figure 3a). Of these records, 424 require one reference call, 119 require two or three, and 60 require at least four. Single-step tasks assess basic data access and tool use, while multistep tasks additionally involve operation composition and intermediate results. Counts refer to calls under the registered interfaces; a tool may perform several operations internally. Source details, construction checks, and cross-theme overlap are described in Appendix C.

4.2 Task Definition and Tool-Call Annotations

We represent an evaluation record as

$$\mathcal{E}_i = (q_i, X_i, y_i^\star, \tau_i^\star), \tag{1}$$

where q_i specifies the scientific question and requested outputs, X_i identifies the input data and resource locators, $y_i^\star$ is the reference answer, and $\tau_i^\star$ is a reference tool-call sequence. Answer types and format requirements support consistent scoring across numerical, categorical, and structured outputs.

The tool-call annotations identify the tool, arguments, and relevant input/output information for each reference step. They connect the question to a concrete analysis over the supplied resources. For example, a task requesting the number of events satisfying a condition can be annotated as loading an event table, filtering its records, and counting the filtered result. This records the intermediate object needed by the final operation, in addition to the expected count.

The reference workflow provides one implementation under the registered interfaces. It supports comparison of tool coverage and order and helps locate differences in execution. Alternative workflows may also produce a valid answer, so agreement with the reference calls is assessed separately from answer correctness.

Construction checks cover source traceability, record completeness, and consistency with registered interfaces. Sample executions check data access and invocation behavior; their validation scope is kept separate from scored agent runs.

4.3 Evaluation Metrics

Scores lie in $[0, 1]$ and are averaged equally over records, including failures. Table 1 reports six scores. FAS measures agreement with the reference answer and keeps partial credit. TSF1 measures whether the successful calls cover the reference tools. PAS measures whether the arguments of aligned calls match the reference. TOS measures whether those calls preserve the reference order. ECR measures the share of attempts that are matched successful calls. Quality is their weighted aggregate. Bash-only agents have no tool trace, so only FAS applies to them. Formulas and weights are in Appendix C.4.

Table 1: GeoToolEval results. All numeric entries are placeholders. Quality is the weighted aggregate in Appendix C.4. Bash-only agents investigate data through shell commands; Quality and tool-chain metrics are not applicable (—).

Model	Quality ↑	FAS ↑	TSF1 ↑	PAS ↑	TOS ↑	ECR ↑
Bash-only agents						
DeepSeek V4.1 Flash	—	0.0000	—	—	—	—
GPT-5.6 Terra	—	0.0000	—	—	—	—
AgentGS						
Qwen3.5-4B	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Qwen3.5-9B	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Qwen3.5-35B-A3B	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Qwen3.8-27B	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Qwen3.8-Plus	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
GeoGPT-Qwen2.5-72B	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
DeepSeek V4.1 Flash	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
GPT-5.6 Terra	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

5 Results

We evaluate AgentGS on GeoToolEval and GeoBenchX, examining answer quality, tool use, and performance across workflow lengths.

Draft note: Numerical comparisons below retain the previous evaluation (Appendix D); the expanded comparison in Table 1, including Quality, PAS, and ECR, remains a placeholder pending reruns.

5.1 Experimental Setup

The GeoToolEval evaluation covers all 603 records, with failures retained in the averages. We report final-answer agreement and tool-workflow metrics defined in Section 4.3. The expanded comparison includes eight AgentGS backbones and Bash-only agents using DeepSeek V4.1 Flash and GPT-5.6 Terra (Table 1). Bash-only agents explore the data through shell commands and are compared by FAS.

The previous evaluation uses GPT-5.6 Terra for AgentGS, GeoAgent [1], and EarthAgent [5], together with direct-answer language-model baselines. Those direct-answer results are distinct from the planned Bash-only comparison. Their scores are retained in Table 7; run provenance and configuration details appear in Appendix D.

5.2 Main Results on GeoToolEval

AgentGS achieves the highest mean FAS among the previously evaluated methods, scoring 0.8731 compared with 0.7804 for GeoAgent and 0.0862 for EarthAgent (Table 7). Its advantage over the strongest agent baseline is 9.27 percentage points. Among direct-answer models, DeepSeek V4 Pro reaches 0.5162, while the same GPT-5.6 Terra backbone without tool execution scores 0.3137. These results support the effectiveness of AgentGS for completing the benchmark's scientific analyses.

The answer advantage is accompanied by higher agreement with reference tool workflows. AgentGS obtains TSF1/TOS of 0.9170/0.9166, compared with 0.8663/0.7911 for GeoAgent and 0.1182/0.1390 for EarthAgent. Relative to GeoAgent, the gains are 5.07 percentage points in tool selection and 12.55 points in relative order. The larger order-score gain is consistent with the role of structured dependencies in organizing execution; Section 6 examines individual mechanisms.

Table 2: AgentGS by reference-chain length on GeoToolEval. Process scores for the single-step stratum are not supplied in the reported breakdown (—).

Reference calls	Records	FAS	TSF1	TOS
1	424	0.9425	—	—
2–3	119	0.7557	0.9779	0.9832
≥ 4	60	0.6167	0.8013	0.7785

Figure 3b presents domain-specific FAS. This breakdown complements the overall score because themes differ in size, data type, and workflow composition. The reported aggregate gains do not imply an equal benefit across all domains.

5.3 Performance across Workflow Lengths

AgentGS obtains FAS of 0.9425 on single-step tasks, 0.7557 on tasks with two or three reference calls, and 0.6167 on tasks with at least four calls (Table 2). Performance decreases as the evaluated workflows become longer.

On the 119 two- to three-step records, TSF1/TOS reach 0.9779/0.9832. On the 60 longer records, they decrease to 0.8013/0.7785. Longer workflows therefore remain challenging in both answer quality and tool orchestration. High process agreement on shorter workflows also coexists with lower FAS, showing that selecting and ordering the expected tools alone does not ensure a correct answer. Domain and length effects overlap: remote sensing accounts for 42 of the 60 longest-chain records.

5.4 External Evaluation on GeoBenchX

We further evaluate AgentGS on GeoBenchX [6] by integrating its official tools and constructing the corresponding Skills and operation mappings. AgentGS and the official ReAct baseline are each

Table 3: Evaluation results on GeoBenchX. Both methods use GPT-5.6 Terra.

Method	Mean score (0–2) ↑	Full-match rate (%) ↑
ReAct	1.1980	53.47
AgentGS	**1.3762**	**62.87**

Table 4: Planned comparison on the shared data-access scope. Numeric entries are placeholders. Quality is not applicable to Data-only (—).

Configuration	FAS	Quality
Data-only (Bash)	0.0000	—
Tool-only	0.0000	0.0000
AgentGS	0.0000	0.0000

reported on 202 tasks using GPT-5.6 Terra. The same model judges reference-solution agreement on a 0–2 scale; we report mean score and the rate of full matches.

AgentGS achieves a mean score of 1.3762, compared with 1.1980 for ReAct, and a full-match rate of 62.87%, compared with 53.47%. The improvements are 0.1782 points and 9.40 percentage points, respectively. Both measures favor AgentGS in this additional tool environment. This evaluation uses dedicated strategy knowledge, and verification of paired task membership and matched conditions remains necessary for a controlled transfer claim (Appendix D).

6 Ablation and Analysis

We examine the contribution of structured strategy knowledge, the mechanisms that construct and complete plans, and their computational cost.

Draft note: The component scores below retain the previous 57-record study. Matched system comparisons, plan-refinement statistics, and cost measurements remain to be populated from reruns.

6.1 From Data to Skills

We separate three levels of agent support. *Data-only* provides the input data and Bash for exploration and computation. *Tool-only* adds the registered scientific tools and their existing planning and execution workflow. *AgentGS* further introduces Skill organization, plan composition, and automatic completion. Data-only versus Tool-only examines the contribution of packaged scientific operations; Tool-only versus AgentGS examines the additional contribution of strategy knowledge and structured planning.

The planned comparison uses the same backbone, questions, accessible data, and model budgets. Tasks depending on data services unavailable to Bash are excluded from all three configurations. FAS provides a common outcome measure; Quality and its process components additionally compare Tool-only with AgentGS. Table 4 reserves these matched results. Earlier direct-answer baselines do not substitute for the Data-only condition.

Table 5: Earlier component comparison on 57 records, including 35 multistep records. A1 is a partial control; A2 did not activate its intended intervention.

Configuration	FAS	Multistep FAS
Full	0.6387	0.5333
A1: Flat planning	0.4748	0.4095
A2: w/o predecessor expansion	0.4760	0.4095
A3: w/o automatic completion	0.4624	0.3802
A4: w/o directed bridging	0.6160	0.4913

6.2 Design Contributions

Within AgentGS, we consider four interventions. *Flat planning* retains strategy declarations and candidate tools but asks the model to plan directly in tool instances. *Without predecessor expansion* retains goal selection and dependency declarations while disabling automatic predecessor closure. *Without automatic completion* disables supporting-operation insertion and bridging. *Without directed bridging* removes only the search for connecting branches, retaining other completion mechanisms. These controls isolate narrower changes than removing the entire Skill layer.

Table 5 retains the earlier results on 57 records, including 35 multistep records. Full obtains overall/multistep FAS of 0.6387/0.5333. Disabling completion gives 0.4624/0.3802, while disabling bridging alone gives 0.6160/0.4913. The corresponding multistep decreases are 15.31 and 4.20 percentage points. These comparisons motivate examining which missing operations completion supplies and when a connecting branch changes the outcome.

Flat planning and the predecessor configuration score below Full, but their earlier runs implemented only a partial control and an inactive intervention, respectively. Their differences cannot yet establish those designs' effects. The reruns must verify effective interventions and changes in the resulting plans. Full process scores, provenance, and supplementary tool-selection and recovery comparisons are retained in Appendix D.5.

6.3 Plan Refinement

The analysis follows the plan from retrieved candidates to selected goals, expanded predecessors, and completed connections. For composition, we will report the number of candidate and retained operations, required-operation coverage, and unresolved inputs. Reference-workflow mappings and inspected traces will check whether pruning preserves necessary operations. A smaller candidate set is useful only when it retains an appropriate route to the requested outputs.

For completion, we will record how often a gap is detected, a supporting operation or branch is inserted, and grounding succeeds. Paired executions will start from the same saved initial plan with completion enabled or disabled, using matched downstream settings and budgets. We will report changes in per-task FAS, corrected and degraded outcomes, and the reasons connections are rejected. Both the full population and the subset with gaps identified before the intervention will be reported.

This comparison tests whether completion repairs an initially incomplete plan rather than benefiting from a different initial model

proposal. Appendix E illustrates the mechanism: a raster-producing fragment and a scalar-comparison fragment are connected through a statistics operation, whose output is explicitly bound to the comparison. The illustration explains the expected repair; its frequency and outcome effects remain to be measured.

6.4 Quality and Cost

We will compare Tool-only, AgentGS, and AgentGS without completion using answer quality, model requests, token consumption, tool attempts, and elapsed time. Measurements will include failed tasks under matched budgets and execution conditions. ECR describes successful reference-call matches per attempt; actual time and token usage are needed to assess computational cost.

Comparing weaker and stronger backbones will test whether structured planning and automatic repair reduce dependence on model capability. The analysis will relate score gains to additional planning calls, inserted operations, and avoided execution failures. No efficiency advantage is inferred from the available accuracy scores alone; these measurements will determine whether improved task completion offsets the extra planning work.

7 Conclusion

We present AgentGS and GeoToolEval for tool orchestration in Earth science. AgentGS separates reusable strategies, shared operations, and concrete implementations through a Skill–Capability–Tool representation. It composes local workflow fragments while retaining their dependencies and usage constraints, then searches the skill hypergraph for admissible operations that complete missing connections. Explicit grounding and artifact bindings connect this plan to executable scientific analysis.

GeoToolEval supplies 603 evaluation records across ten research themes, with reference answers and tool workflows that support separate measures of answer agreement, tool coverage, and invocation order. AgentGS achieves reported mean FAS of 0.8731, compared with 0.7804 for the strongest reported agent baseline, and also obtains higher aggregate process scores. The configuration study exposes answer and workflow differences while identifying limits on attributing them to individual mechanisms.

The resulting framework makes composition and completion inspectable at the level of operations, dependencies, and observed artifacts. Further evaluation should examine unseen capability combinations, controlled changes in the tool environment, and the utility of verified compositions as reusable strategies. Broader scientific reliability also requires validating task semantics and resource use beyond agreement with a reference call sequence.

References

[1] Yuxing Chen, Weijie Wang, Sylvain Lobry, and Camille Kurtz. 2024. An LLM Agent for Automatic Geospatial Data Analysis. arXiv:2410.18792 doi:10.48550/arXiv.2410.18792

[2] Cheng Deng, Tianhang Zhang, Zhongmou He, Yi Xu, Qiyuan Chen, Yuanyuan Shi, Luoyi Fu, Weinan Zhang, Xinbing Wang, Chenghu Zhou, Zhouhan Lin, and Junxian He. 2023. K2: A Foundation Language Model for Geoscience Knowledge Understanding and Utilization. arXiv:2306.05064 doi:10.48550/arXiv.2306.05064

[3] Gabriel Diaz-Ireland, Diego Prieto-Herráez, Mario García Peces, Javier Velázquez, and Devika Jain. 2026. GeoNatureAgent Benchmark: Benchmarking LLM Agents for Environmental Geospatial Analysis Across Frontier and Open-Weight Foundation Models. arXiv:2606.12821 doi:10.48550/arXiv.2606.12821

[4] Keyan Ding, Jing Yu, Junjie Huang, Yuchen Yang, Qiang Zhang, and Huajun Chen. 2025. SciToolAgent: A Knowledge Graph-Driven Scientific Agent for Multi-Tool Integration. arXiv:2507.20280 doi:10.48550/arXiv.2507.20280

[5] Peilin Feng, Zhutao Lv, Junyan Ye, Xiaolei Wang, Xinjie Huo, Jinhua Yu, Wanghan Xu, Wenlong Zhang, Lei Bai, Conghui He, and Weijia Li. 2026. Earth-Agent: Unlocking the Full Landscape of Earth Observation with Agents. In *International Conference on Learning Representations*. https://arxiv.org/abs/2509.23141v3

[6] Varvara Krechetova and Denis Kochedykov. 2025. GeoBenchX: Benchmarking LLMs in Agent Solving Multistep Geospatial Tasks. In *Proceedings of the 1st ACM SIGSPATIAL International Workshop on Generative and Agentic AI for Multi-Modality Space-Time Intelligence*. ACM, 27–35. doi:10.1145/3764915.3770721

[7] Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. 2023. LLM+P: Empowering Large Language Models with Optimal Planning Proficiency. arXiv:2304.11477 doi:10.48550/arXiv.2304.11477

[8] Yanming Liu, Xinyue Peng, Jiannan Cao, Shi Bo, Yuwei Zhang, Xuhong Zhang, Sheng Cheng, Xun Wang, Jianwei Yin, and Tianyu Du. 2025. Tool-Planner: Task Planning with Clusters across Multiple Tools. In *International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2025/file/7f605d59a0dbde101518b552cb616ddf-Paper-Conference.pdf

[9] Felipe Meneguzzi, Ramon Fraga Pereira, and Nir Oren. 2025. Generalised BDI Planning. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, 1483–1491. doi:10.5555/3709347.3743782

[10] Qinzhuo Wu, Wei Liu, Jian Luan, and Bin Wang. 2024. ToolPlanner: A Tool Augmented LLM for Multi Granularity Instructions with Path Planning and Feedback. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 18315–18339. doi:10.18653/v1/2024.emnlp-main.1018

[11] Wanghan Xu, Xiangyu Zhao, Yuhao Zhou, Xiaoyu Yue, Ben Fei, Fenghua Ling, Wenlong Zhang, and Lei Bai. 2025. EarthSE: A Benchmark for Evaluating Earth Scientific Exploration Capability of LLMs. arXiv:2505.17139 doi:10.48550/arXiv.2505.17139

[12] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*. https://arxiv.org/abs/2210.03629v3

[13] Bo Yu, Cheng Yang, Dongyang Hou, Chengfu Liu, Jiayao Liu, Chi Wang, Zhiming Zhang, Haifeng Li, and Wentao Yang. 2026. GeoAgentBench: A Dynamic Execution Benchmark for Tool-Augmented Agents in Spatial Analysis. arXiv:2604.13888 doi:10.48550/arXiv.2604.13888

[14] Zian Zhai, Xingyu Tan, Gaowang Zou, Xiaoyang Wang, and Wenjie Zhang. 2026. HyperAgent: Planning and Acting over Tool-Schema Hypergraphs for Tool-Use LLM Agents. arXiv:2608.02650 doi:10.48550/arXiv.2608.02650

[15] Yongfei Zheng. 2025. Research Forefronts in Earth Science. *Chinese Science Bulletin* 70, 36 (2025), 6188–6203. In Chinese. doi:10.1360/CSB-2025-5248

A Representation and Completion Details

This appendix provides the formal representation, dependency closure, and branch acceptance rules supporting Section 3.

A.1 Representation and Plan Notation

Skill hypergraph. Let S and C denote the registered Skills and Capabilities. Their membership defines

$$H = (C, E), \qquad E = \{e_s : s \in S\}, \qquad e_s \subseteq C. \tag{2}$$

Capabilities are nodes and each Skill is a hyperedge. Skills containing $\{a, b\}$ and $\{b, c\}$ intersect at capability b, providing a candidate connection for composition. Hyperedge membership records which operations belong to a strategy. It does not, by itself, specify the order or data flow between them.

Execution prerequisites are represented separately by a scoped relation $d = (s, U, c)$: within Skill s, capability c requires every predecessor in U. This is a conjunctive requirement. When an operation requires both a prepared raster and a reference value, supplying either input alone does not satisfy the declaration. Conditional order is used only when its stated condition applies. The strategy identity remains part of each relation, preventing a dependency from one Skill from being silently substituted for another's.

Capability instances and artifact bindings. A task-specific plan is

$$P = (V, D_d, D_c, \beta), \qquad v_i = (c_i, s_i, r_i), \tag{3}$$

where c_i is the operation, s_i its source Skill, and r_i its step/input role. D_d and D_c represent data and control dependencies, respectively; β binds inputs to task resources or outputs of preceding instances. A capability is a reusable definition, whereas an instance is one occurrence on particular inputs. Two operations on different rasters remain distinct even if they select the same tool. This distinction is essential for preserving multiplicity, provenance, and the identity of the artifact consumed downstream.

A.2 Mandatory-Predecessor Expansion

Selected operations can depend on outputs not yet included in the plan. For a selected Skill s, let $R_s^{(0)}$ contain its initially required capabilities. Its dependency closure can be written as

$$R_s^{(k+1)} = R_s^{(k)} \cup \bigcup_{\substack{(s,U,c)\in\mathcal{D} \\ c\in R_s^{(k)}}} U, \tag{4}$$

where $\mathcal{D}$ is the set of declared mandatory dependencies. Expansion continues until no required predecessor is missing. This expression describes closure within a strategy; instantiation still preserves separate occurrences and input roles. Missing declarations and dependency cycles are diagnosed rather than accepted as executable plans.

A.3 Directed Bridging with a Fixed Active Boundary

Suppose activated Skills A and B supply disconnected fragments. Let u be an output-boundary capability of A, with no successor among its active dependencies, and v an input-boundary capability of B, with no predecessor among its active dependencies. If inactive Skill C declares $u \to m \to v$, this branch can supply intermediate capability m without importing the entire Skill.

The active capability set is fixed at search start. Each branch must come from a single inactive Skill and contain at least one inactive intermediate capability. Membership first identifies Skills containing both endpoints; explicit dependencies or applicable conditional-ordering relations then determine directed simple paths. Co-occurrence and list order do not establish direction. An existing indirect data path from u to v excludes the candidate pair; a direct connection alone does not. Newly inserted capabilities do not recursively extend the current search.

Candidate branches must satisfy five checks:

(1) **Direction:** each adjacent pair follows an explicit dependency or applicable conditional order.
(2) **Prerequisites:** all required predecessors belong to the fixed active set or candidate branch.
(3) **Types:** upstream outputs and downstream inputs share at least one concrete type, excluding generic labels such as `Any` and `Parameter`.
(4) **Implementations:** branch operations and required predecessors have available tools within permitted scopes. The entry and external active predecessors retain their original scopes; other branch nodes use the provider Skill's constraints.
(5) **Acyclicity:** both individual and joint insertions preserve acyclic dependencies.

For each ordered endpoint pair, only a unique valid branch may be inserted automatically. Multiple candidates produce an ambiguity report and halt completion for that pair. Insertion preserves the entry instance's dependencies and tools, updates the exit's dependencies, tool scope, and bindings, and recomposes the plan. A branch is rejected if recomposition adds capabilities outside the branch or leaves mandatory dependencies unsatisfied. Algorithm 1 summarizes the procedure.

Algorithm 1 Directed Bridging from a Single Provider

Require: Plan P after compatibility completion; registered knowledge; available tools
Ensure: Updated P and acceptance/rejection records
1: Return if bridging is disabled or this plan has already been searched
2: Fix active capabilities; mark P as searched
3: Enumerate boundary pairs from different Skills; skip pairs with an indirect data path
4: Search inactive Skills for single-provider branches satisfying the five validity checks
5: Deduplicate by provider/path; reject ambiguous pairs; check joint acyclicity
6: Attempt insertion, preserving entry constraints and updating exit dependencies, tools, and bindings
7: Recompose; reject external additions or unmet mandatory dependencies

A.4 Grounding an Accepted Branch

A capability-level branch must be projected onto execution instances before it can carry data. The entry and exit must identify unique instances with an order consistent with the bridge. Required operations are inserted with their scoped implementations and all conjunctive predecessor dependencies. An endpoint ambiguity is a grounding failure, not permission to choose an arbitrary occurrence.

Each declared bridge input identifies an upstream instance and a projection from its observed output. Projection can select a named object field or a nonnegative array index; a declared list transformation can wrap an artifact as a list input. A producing step must succeed before the value becomes available. If the branch cannot be grounded consistently, the system rejects that projection rather than treating capability connectivity as executable data flow.

A.5 Candidate Skill Representation

A verified composition can be described as a candidate Skill containing its objective, applicable inputs, expected outputs, capability instances, data and control dependencies, permitted tools, and usage constraints. Task-specific paths and values become rebindable inputs where appropriate. Distinct occurrences remain distinct, and provenance links retain the conditions inherited from contributing strategies.

Registration requires checking those declarations and assigning the candidate a strategy identity before making it available for later retrieval. The proposal reuses existing capabilities and implementations; it does not create new scientific operations. Candidate packaging and registration are an extension of the workflow representation. The reported experiments do not evaluate autonomous registration, long-term accumulation, improved coverage, or reduced planning cost.

B System Architecture and Execution Details

AgentGS implements the method through an interactive frontend, an orchestration service, registered knowledge resources, and a tool execution service (Figure 4). This appendix describes how the plan in Section 3 becomes an inspectable sequence of calls.

B.1 Separation of Planning and Execution

The frontend accepts questions and resources and displays the returned answer and execution records. Orchestration retrieves strategies and tools, prepares capability requirements, invokes model planning, and validates the resulting instances. Knowledge resources provide Skill declarations, capability mappings, and tool descriptions. The execution service invokes a registered interface and returns its observed output or failure.

This separation allows planning to reason over declared operations while execution operates on concrete argument objects. Descriptive similarity between two tools does not by itself authorize substitution: the capability occurrence's scope and the selected interface schema still apply.

B.2 Execution Instances and Input Bindings

An execution step records its identity, tool, purpose, predecessor identities, literal arguments, and upstream input bindings. Optional Skill and Capability identifiers establish its declared scope. Unknown or forward dependencies and duplicate instance identities are rejected before execution.

A binding names a producing instance and a path into its observed structured output. The consumer receives the resolved value rather than an unverified description of what the producer was expected to return. Explicit references remain authoritative when arguments are repaired. This prevents a failed or inappropriate argument from being silently replaced with a convenient unrelated value.

Artifacts additionally retain provenance such as the producing step and tool. File and scalar bindings are validated according to their expected inputs. An operation on the wrong artifact may still execute successfully, which is why invocation success and reference-answer correctness are tracked separately.

B.3 Recovery and Reuse Boundaries

When execution recovery is enabled, the runtime can use failure feedback for argument correction and another plan attempt under configured limits. Initial model-plan repair, protocol repair, transport retries, and scientific tool retries have separate counters and scopes. Exhausting a communication budget does not establish that a scientific computation has been safely replayed.

A result can be reused across plan attempts only when the instance has unchanged semantics, fully specified inputs, and verified upstream reuse identities. The current runtime permits this for an explicit set of pure scalar computations. File-backed, implicit-input, and external-data operations are rerun rather than assumed unchanged. Matching tool names or retaining the same position in a chain is insufficient to establish reuse.

With execution recovery disabled, the first failed scientific execution ends that chain and answer generation reports incomplete work. Initial argument construction and validation remain available, so the intervention targets reaction to observed failure rather than removal of ordinary execution checks.

B.4 Knowledge Integration and Process Inspection

Integrating a tool registers its identifier, parameter schema, and input/output information, then associates it with relevant capabilities. A Skill supplies the strategy-specific dependency and implementation constraints. Runtime selection checks those declarations together with tool availability and the task's concrete inputs.

The interface exposes plans, invocation inputs and outputs, execution states, and failure feedback. Tool-graph browsing presents registered tools and associations as retrieval and inspection aids. These associations do not supersede capability dependencies or establish valid argument flow. Retained histories make it possible to inspect whether the intended file was read, whether the correct upstream value was consumed, and which attempts contributed to the final answer.

B.5 Grounding, Execution, and Verification

A completed capability plan describes operations and dependencies. Grounding converts it into execution instances with concrete tool identifiers, arguments, and references to upstream results.

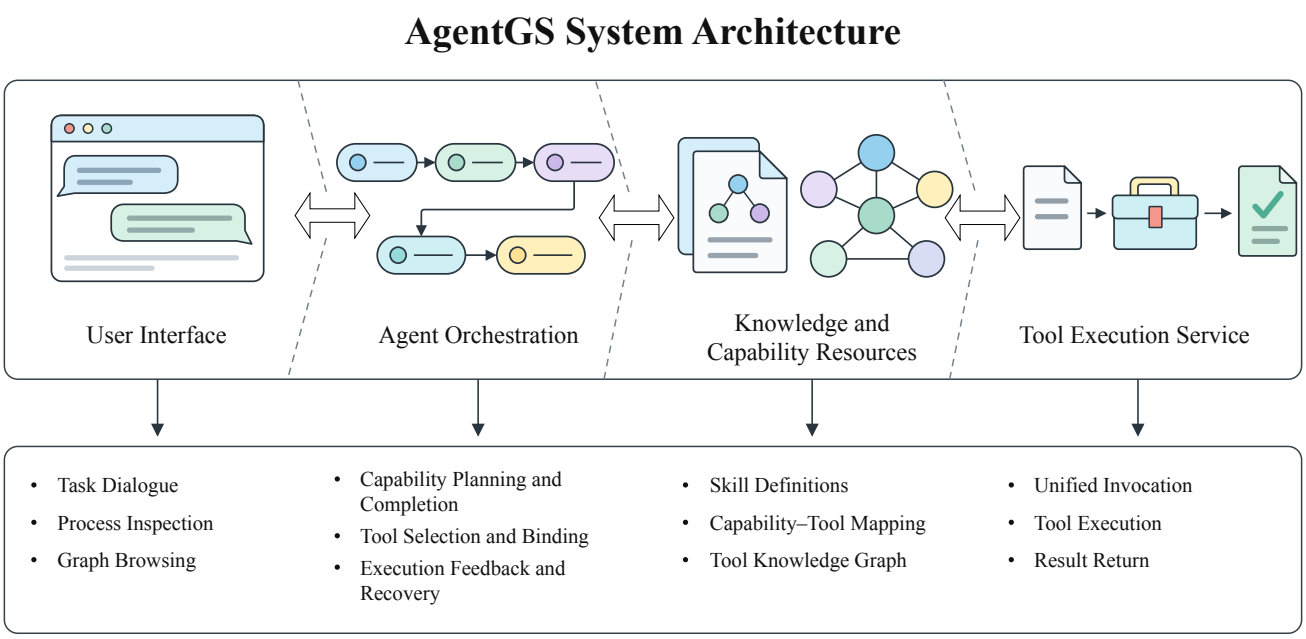


Figure 4: AgentGS architecture and main functions. The system comprises an interactive frontend, an agent orchestration service, knowledge and capability resources, and a tool execution service.

Constrained tool selection. For each capability instance, the candidate set is restricted by its source Skill, operation requirements, tool availability, and interface constraints. GeoToolKG supplies tool descriptions and associations for retrieval; those associations do not override the plan's dependencies. The LLM selects implementations within the admitted scope. Required operations and scoped ordering are checked again after the concrete chain is produced.

Explicit input propagation. Each execution instance has an identity and declares its predecessors. Input bindings name a producing instance and a field in that instance's observed output. At execution time, the system resolves these references and validates the resulting arguments against the selected tool's schema. A consumer cannot substitute an unrelated earlier call merely because it used the same tool name. Unknown predecessors, unresolved references, and incompatible arguments prevent invocation.

This separation is especially important for scientific artifacts. In a compositing task, the statistics call must consume the generated composite, not another raster mentioned in the question. Likewise, a comparison must use the statistic actually returned by its producer. The execution record links these inputs to their sources, making artifact use inspectable independently of the final answer.

Failure handling. Execution failures produce structured feedback for bounded argument repair or replanning when recovery is enabled. Literal arguments may be corrected, while observed upstream bindings remain authoritative. Replanning validates instance identities, dependencies, and scopes again. Reuse of a successful result requires unchanged inputs and instance semantics; it is not inferred from a common tool-name prefix. The runtime conservatively reruns file-backed and external-data computations across plan attempts, while allowing reuse of explicitly supported pure scalar computations when their complete dependencies are verified.

Tool recovery, initial plan repair, and transport retries have separate roles and budgets. Disabling execution recovery terminates the scientific chain after an execution failure while retaining initial planning and input validation. Engineering details and the distinction between these budgets appear in Appendix B.

Verification and answer generation. The answer is constructed from observed results and artifacts. Verification checks whether the requested outputs are supported and records unfinished operations when execution is incomplete. Successful invocation, evidence consistency, and agreement with a benchmark reference are distinct properties; the benchmark evaluates the last of these independently.

A verified composition can also be represented as a candidate reusable Skill, preserving its input roles, dependencies, and usage constraints. This is a strategy representation over existing operations, not an expansion of the underlying tool collection. The present results do not establish a performance or cost benefit from autonomous Skill accumulation; the candidate-registration design is described in Appendix A.5.

Table 6: GeoToolEval task sources and reference tool-invocation chain lengths. Counts refer to evaluation records.

Task theme	Main data or question sources	1 step	2–3	≥ 4	Total
Three-dimensional Earth structure	LITHO1.0, SEMUCB-WM1, USGS earthquake catalog	85	15	0	100
Geoscientific big-data analysis	Selected GeoQuestions1089 subset	40	0	0	40
Critical minerals	Habitable-Earth questions; MRDS and cross-theme climate data	44	14	0	58
Climate change	Habitable-Earth questions; paleoclimate, GISTEMP, SSP, and cross-theme mineral data	31	16	1	48
Energy resources	MRDS, GEM, OPSD, World Bank, and population data	100	0	0	100
Global ocean carbon cycle	SOCAT, GLODAP, SNAPO-CO_2	100	0	0	100
Deep-time biosphere	BioDeepTime	12	7	5	24
Carbon capture and storage	Synthetic SCCS-OFPOSDU CCS–MRV data	12	5	7	24
Remote sensing	Selected Earth-Bench subset	0	7	42	49
Natural-disaster event statistics	EM-DAT International Disaster Database	0	55	5	60
Total		**424**	**119**	**60**	**603**

C Dataset Sources and Construction Records

C.1 Source-Specific Task Construction

Geoscientific big-data analysis includes 40 GeoQuestions1089 questions concerning physical geographic objects and spatial operations. The remote-sensing theme retains 49 tasks from Earth-Bench's 248-task collection, including source questions, choices, and answers, with calls adapted to the registered interfaces. Critical-mineral and climate-change questions are selected by theme from habitable_earth_dataset; the underlying resources include MRDS, paleoclimate reconstructions, GISTEMP, and SSP warming summaries.

Earth-structure tasks use two models and an earthquake catalog. The models are LITHO1.0 and SEMUCB-WM1; earthquake records come from USGS. Energy-resource tasks use mineral, power-facility, installed-capacity, and population data. Global ocean-carbon tasks draw on SOCAT, GLODAP, and SNAPO-CO_2. Deep-time biosphere tasks use BioDeepTime, carbon-storage tasks use synthetic SCCS-OFPOSDU monitoring data, and disaster-event tasks use EM-DAT records. Synthetic carbon-storage data support reproducible operations under their supplied assumptions; they do not constitute field validation of storage safety.

C.2 Evaluation Records and Provenance

The dataset's task representation preserves one worksheet row per evaluation record. Fields identify the question and category, question text, expected answer, reference-chain information, answer type, input resource, and answer-format requirements. Stable record identities distinguish evaluation entries even when a question appears in two themes.

A source task retains its question identifier or resource locator and its reference answer. A data-derived task retains data-location information and calculations describing how the reference was obtained. Reference calls map these operational requirements to registered tools, including parameters and the relevant input/output relationships.

Construction checks record completeness and correspondence between reference-call names and registered interfaces. Sample executions check data access and invocation behavior, while computational records support recalculation. These checks are recorded separately from scored agent runs, preventing a successful construction check from being counted as an agent result.

C.3 Interface Granularity and Aggregation

The reference sequence is defined under the tool packaging available to the evaluated agents. A task wrapper may hide several operations inside one call, while another interface exposes them separately. Consequently, invocation count measures a property of the exposed workflow rather than an intrinsic scientific complexity.

Record-weighted averaging gives each of the 603 entries equal weight. A domain-balanced mean would instead give the ten themes equal weight, and a deduplicated question mean would change the treatment of repeated cross-theme questions. Those quantities answer different questions and are not interchangeable with the reported main FAS.

Numerical tolerances and answer normalization belong to the answer-type-specific scorer. Process metrics do not validate those numerical decisions. Conversely, matching a reference answer does not establish that the agent produced every requested artifact or followed the intended data dependencies.

The critical-mineral and climate-change themes share nine questions, represented in both themes. The 603 records therefore do not denote 603 distinct question texts. Remote sensing accounts for 42 of the 60 records with at least four reference calls, so domain and chain-length effects overlap in this dataset.

C.4 Metric Definitions and Aggregation

Let G_i be the reference sequence of tool names and A_i the sequence of successful calls in the current user turn. Calls match one to one by tool name, retaining multiplicity, and m_i is the number of matched occurrences. Failed calls stay out of A_i. Planning calls are excluded from both successful calls and attempts. A score whose denominator is zero is defined to be zero. For any per-record score M and N records,

$$\overline{M} = \frac{1}{N} \sum_{i=1}^{N} M_i. \tag{5}$$

The main experiment uses $N = 603$. Domain and chain-length groups average their own records. This record-weighted mean is distinct from a domain-balanced macro-average and from the rate of fully correct answers.

FAS follows the answer type: choice letters, calendar dates, identifiers, numerical tolerances, item matching, or field comparison. An empty answer scores zero.

Tool selection is the F1 of successful-call precision and reference-call recall:

$$\mathrm{TSF1}_i = \frac{2m_i}{|G_i| + |A_i|}. \tag{6}$$

Missing tools, extra successful calls, and repeated successful calls change the score.

Let π_i be a longest-common-subsequence alignment from reference steps to calls in A_i. For reference step k, the argument similarity $s_{i,k}$ is zero when k is not aligned. Otherwise $s_{i,k} = \mathrm{Sim}(e_{i,k}, a_{i,\pi_i(k)})$, where $e_{i,k}$ is the reference argument and $a_{i,\pi_i(k)}$ is the argument of the aligned call. Equal normalized values score 1, as do scalars within a numeric tolerance. Mappings and sequences score the F1 of their recursively matched fields or items. A reference argument that is an intermediate result is compared with the value produced by the preceding successful call that supplies it. Parameter alignment is

$$\mathrm{PAS}_i = \frac{1}{|G_i|} \sum_{k=1}^{|G_i|} s_{i,k}. \tag{7}$$

Tool order is the normalized longest common subsequence:

$$\mathrm{TOS}_i = \frac{\mathrm{LCS}(G_i, A_i)}{|G_i|}. \tag{8}$$

For the reference order read–filter–count and the executed order read–count–filter, TSF1 is 1 and TOS is 2/3. Extra calls do not necessarily lower TOS.

Let U_i be the attempt sequence, including failed calls and excluding planning calls. Execution efficiency is the fraction of those attempts that are matched successful calls:

$$\mathrm{ECR}_i = \frac{m_i}{|U_i|}. \tag{9}$$

Quality applies fixed weights, with the largest weight on the answer and the smallest on order and attempt efficiency:

$$\begin{aligned}\mathrm{Quality}_i = {} & 0.4\,\mathrm{FAS}_i + 0.2\,\mathrm{TSF1}_i + 0.2\,\mathrm{PAS}_i \\ & + 0.1\,\mathrm{TOS}_i + 0.1\,\mathrm{ECR}_i.\end{aligned} \tag{10}$$

A reference chain records one solution under the available interfaces. An alternative valid implementation can receive lower tool-name scores. PAS compares arguments only on steps aligned by tool name, so an unaligned call contributes no argument credit. These scores measure agreement with the reference workflow rather than semantic equivalence of arbitrary analyses. Explaining an answer or judging an alternative workflow still requires the inputs, outputs, and dependency trace.

Table 7: Earlier GeoToolEval baseline comparison. AgentGS process metrics are per-record averages from the present 603-record end-to-end experiment; FAS values are retained from the source manuscript. Direct-answer language models have no tool traces; their process metrics are not applicable (—).

Method	FAS	TSF1	TOS
AgentGS (GPT-5.6 Terra)	**0.8731**	0.9170	0.9166
GeoAgent (GPT-5.6 Terra)	0.7804	0.8663	0.7911
EarthAgent (GPT-5.6 Terra)	0.0862	0.1182	0.1390
DeepSeek V4 Pro	0.5162	—	—
GPT-5.6 Terra	0.3137	—	—
GPT-5.4	0.3069	—	—
Qwen-Plus	0.2918	—	—
GeoGPT	0.2500	—	—

D Experimental Provenance and Control Details

D.1 Sources of the Reported Aggregates

Table 7 retains the answer scores supplied by the earlier experiment, with AgentGS identified as its first complete 603-record run. AgentGS's TSF1/TOS are supplied by the recorded end-to-end evaluation on that population. Combining these aggregates in one table does not establish paired answer/process outcomes for every task.

The 57-record configuration study has a separate Full reference. Full's process scores are taken from the supplementary experiment, while A1–A6 use per-task records in the formal worksheet. A1's partial control, A2's inactive intervention, and A5's partial ranking change apply to those reported runs. Current implementation of a corresponding switch does not validate the identity or behavior of a past run.

GeoBenchX supplies two 202-task aggregates and uses GPT-5.6 Terra as both backbone and single reference-answer judge. The exact paired task identities and matched execution settings are not established by the aggregate table. Its 0–2 scoring scale must remain separate from GeoToolEval's answer-type-specific FAS.

The available manuscript evidence does not provide a complete set of immutable model snapshots, prompts, sampling settings, per-task budgets, source revisions, and run identifiers for all comparisons. Those values are not inferred from the current runtime defaults. The present tables report available measurements without substituting newly implemented controls for unverified historical conditions.

D.2 Interventions in the Current Runtime

Module interventions operate within Skill-guided planning. A1 preserves access to strategy declarations and admitted capabilities while requesting tool-instance planning without an ordered capability plan. This differs from a tool-only baseline that removes the Skill/Capability layer altogether.

A2 disables automatic predecessor closure during preparation and support completion. Support insertion does not restore omitted predecessors. A3 suppresses supporting-capability insertion and

bridging, while retaining gap diagnosis. With A2 and A3 together, neither automatic insertion phase supplies missing dependencies. The model may still explicitly include declared requirements. Effective intervention therefore needs to be evaluated through changes in preparation and planning, not assumed from a configuration flag.

A4 suppresses only directed bridging. A5 selects the first eligible implementation in a fixed tool-identifier order, preserving instance identities, constraints, and argument checks. A6 suppresses execution-driven repair and replanning after a scientific failure. These definitions specify intended controlled comparisons; the reported A1, A2, and A5 measurements retain the limitations stated above.

Effective settings, requested interventions, and suppressed controls are recorded with execution provenance. A resume operation must preserve the same experiment identity. These records support checking whether a run applies its requested intervention but do not establish a nonzero effect on a task with no relevant dependency, alternative tool, or recoverable failure.

D.3 Candidate Access, Budgets, and Additional Baselines

A comparison of planning representations should specify whether all systems receive the same tool candidates or retain their own retrieval policies. The runtime supports exposing all registered candidates, while the retrieved setting preserves system-specific candidate selection. Routing hints and binding assistance are separate controls and can remain active even when heuristic plan generation is disabled.

A Bash-based data-only baseline changes the execution interface more substantially. A comparable data-access scope excludes materialized-query lookup tasks whose snapshot service is unavailable in that environment. A comparison that uses the full tool population for one system and a restricted accessible population for another must not interpret the difference as a planning effect. No new Bash-baseline score is asserted here.

Budgets should distinguish model requests, transport attempts, plan attempts, scientific executions, and diagnostic calls. The runtime records separate counters, but the supplied result tables do not include matched latency, token usage, or execution-cost measurements. Consequently, they establish no quantitative cost advantage.

D.4 Trace-Level Analyses Not Established by Aggregates

To relate composition to outcomes, a trace must show the selected requirements, inserted predecessors, supporting operations, and remaining gaps. To relate bridging to outcomes, it must show the provider, endpoints, accepted path, and resolved data bindings. To relate recovery to outcomes, it must distinguish failed attempts, repaired arguments, changed plans, and verified reuse.

The illustrative case provides a concrete explanation of these relationships. It is not a sampled failure distribution or a measured estimate of component utilization. Repeated matched runs and per-task alignment are needed before reporting uncertainty, significance, or causal component effects.

D.5 Complete Component Results

Tables 8 and 9 retain the complete earlier configuration comparisons. The subset covers all ten domains, with 22 single-step, 20 two- to three-step, and 15 longer records. Its multistep fraction is 61.40%, so it differs from the main evaluation population. The recorded A1, A2, and A5 intervention limitations remain applicable to these numbers.

Full's overall TSF1/TOS are 0.9249/0.9392, and its multistep scores are 0.8862/0.8952. A3 obtains multistep TSF1/TOS of 0.8364/0.8438, while A4 obtains 0.8249/0.8652. The higher order score but lower coverage score of A4 illustrates why the two measures should be interpreted separately. These are configuration-level differences rather than an additive decomposition of component effects.

A5 changes only part of tool ranking in the reported run. Its overall FAS rises from Full's 0.6387 to 0.6415, while multistep FAS falls from 0.5333 to 0.5184. Its multistep TSF1/TOS are 0.8133/0.8414. The directions therefore differ across the answer-score populations.

A6 disables execution recovery and obtains overall/multistep FAS of 0.5972/0.4613. Its multistep TSF1/TOS are 0.7652/0.7985, decreases of 12.10/9.67 percentage points from Full. Attributing these differences to recovered failures requires matched execution traces. Recovery is part of the shared execution framework, so this comparison is supplementary to the main planning interventions.

E Illustrative Case Study

We use an illustrative case to show how the system combines capabilities from different Skills, fills missing steps, and forms concrete tool calls. Consider the task: given several remote-sensing rasters of the same region, create and return a pixel-wise mean composite, compute the mean of its valid pixels, state whether that mean exceeds a given reference value, and report the difference. The inputs represent the same variable, their spatial grids are aligned, and the reference value uses the same units as the raster values.

Skill Retrieval returns only RasterCompose (Skill A) and ReferenceCompare (Skill B). *Skill Planning* selects InputCheck (w) and RasterMean (u) from the former, and ScalarDiff (v) and SignCheck (z) from the latter. InputCheck verifies the variable and spatial-grid requirements. RasterMean produces the mean raster from the validated image list, giving $w \rightarrow u$. ScalarDiff returns the difference between the value under comparison and the reference; SignCheck uses that difference to determine whether the value is above, equal to, or below the reference, giving $v \rightarrow z$. Hypergraph Composition expands mandatory predecessors declared by the selected strategies and retains provenance and candidate-tool scopes. At this stage, the plan consists of two disconnected fragments: compositing produces a raster file, whereas differencing requires a numerical input. The operation computing the mean of the composite is still missing.

Using u and v as shared endpoints, Missing Capability Completion retrieves $u \rightarrow m \rightarrow v$ from inactive RasterDiff (Skill C), where ValidMean (m) extracts the mean of valid pixels from the composite. Here, RasterDiff means differencing a statistic of the composite against a reference value. The query explicitly requires the mean statistic; Skill C supplies reusable dependencies and usage constraints connecting it to the preceding and following operations. Under this example's assumptions, it is the only candidate for the ordered endpoint pair satisfying prerequisite, type, tool-availability,

Table 8: Complete earlier configuration comparisons on the fixed 57-record subset. A1 is a partial control, A2 did not activate the intended ablation, and A5 changes only part of tool ranking. ΔFAS is the full method's mean FAS minus the corresponding configuration's mean FAS, in percentage points; positive values indicate lower performance than the full method.

Configuration	Mean FAS	Multistep FAS	ΔFAS (pp)
Full method	0.6387	0.5333	—
A1: Direct tool-level planning	0.4748	0.4095	16.39
A2: No predecessor expansion	0.4760	0.4095	16.27
A3: No independent completion	0.4624	0.3802	17.63
A4: No directed bridging	0.6160	0.4913	2.27
A5: No tool-selection optimization	0.6415	0.5184	−0.28
A6: No execution recovery	0.5972	0.4613	4.15

Table 9: Ablation process metrics. Multistep scores cover the 35 subset records with at least two reference calls. Each column averages per-task scores over its corresponding records. Full uses the supplementary 57-task experiment; A1–A6 use per-task records from the formal 57-task worksheet.

Configuration	Mean TSF1	Mean TOS	Multistep TSF1	Multistep TOS
Full method	0.9249	0.9392	0.8862	0.8952
A1: Direct tool-level planning	0.8599	0.8778	0.8147	0.8295
A2: No predecessor expansion	0.8694	0.8958	0.8302	0.8588
A3: No independent completion	0.8732	0.8865	0.8364	0.8438
A4: No directed bridging	0.8603	0.8997	0.8249	0.8652
A5: No tool-selection optimization	0.8503	0.8851	0.8133	0.8414
A6: No execution recovery	0.8208	0.8588	0.7652	0.7985

and acyclicity requirements: u's raster output is compatible with m's input, and m's numerical output is compatible with v. Insertion preserves u's original constraints, adds m, and updates v's dependencies, candidate-tool scope, and input bindings. Existing dependencies $w \rightarrow u$ and $v \rightarrow z$ remain. Recomposition and dependency verification connect the fragments into input validation, mean compositing, valid-pixel mean statistics, scalar differencing, and sign comparison: $w \rightarrow u \rightarrow m \rightarrow v \rightarrow z$.

Tool Grounding then selects validation, compositing, statistics, differencing, and sign-comparison tools within each instance's candidate scope and parameter constraints. GeoToolKG associations guide candidate retrieval, but concrete invocations still require instance and input-compatibility checks. Grounding a bridge into calls additionally requires that u and v each correspond to exactly one endpoint instance and that their order agrees with the bridge direction. During execution, the validated image list is passed to the compositing tool. The statistics tool receives a list containing only the composite raster's path. The returned mean and the task's reference value are bound to differencing parameters a and b, producing $d = a - b$. Sign comparison then compares d with zero. The statistical object remains the composite created for this task, the comparison baseline remains the supplied reference, and the final relation is based on the same difference calculation.

If the statistics call fails, recovery can repair the invocation using the observed failure and the applicable scope. A local retry can continue to refer to the composite produced earlier in the current plan attempt. A new plan attempt does not automatically reuse file-backed computations: their state must be re-established according to the execution policy. This distinction prevents the illustration from implying an unrestricted cache of prior artifacts.

Successful execution yields the composite raster, its valid-pixel mean, the numerical difference, and the comparison relation. These outputs must refer to the same composite and reference value. Merely selecting the five expected tool names would not establish that consistency; the artifact bindings are the operational link.

After verification, the composition can be described as a candidate CompositeCompare Skill (D), parameterizing the image list and reference value while retaining dependencies and usage constraints. The example illustrates how a new strategy can be represented over existing capabilities. It does not provide an empirical result on registration, reuse across tasks, or improved planning efficiency.

F Limitations and Further Evaluation

F.1 Dependence on Registered Knowledge

Composition depends on the coverage and quality of Skill declarations, dependency relations, and implementation mappings. A hypergraph can connect known operations but cannot supply a scientific operation absent from the registry. Incomplete or inaccurate constraints can prevent useful reuse or admit an unsuitable composition. Dedicated knowledge construction for GeoBenchX is part of the evaluated deployment and should be included when considering the cost of adopting another tool environment.

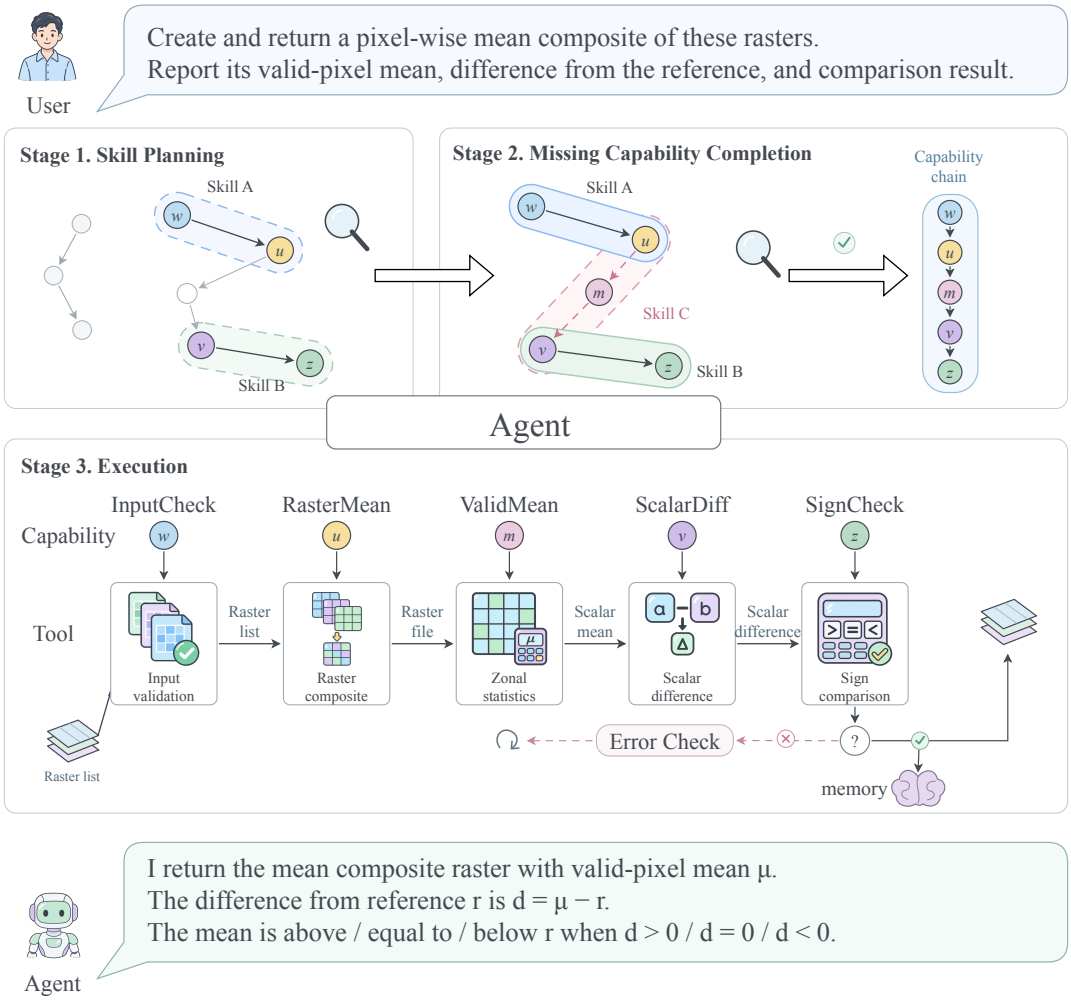


Figure 5: Cross-Skill capability composition and tool execution. Stage 1 selects $w \to u$ from Skill A and $v \to z$ from Skill B. Stage 2 inserts valid-pixel mean statistics m using Skill C's $u \to m \to v$ branch. Stage 3 grounds the five-step workflow in tool calls, propagates intermediate artifacts, and returns the composite raster, mean, difference, and comparison relation. The workflow is a mechanism illustration.

F.2 Structural Validity and Scientific Meaning

Type compatibility and dependency closure establish declared workflow conditions. They do not prove that a selected statistic, spatial resolution, physical model, or comparison baseline is scientifically appropriate. Scientific validation also depends on data quality, units, assumptions, and the meaning of the question. The framework makes bindings and dependencies inspectable, but expert or domain-specific checks can still be necessary.

F.3 Benchmark Scope and Metric Sensitivity

GeoToolEval has unequal theme sizes, repeated cross-theme questions, and interface-dependent chain lengths. Its record-weighted mean therefore reflects the chosen population and wrappers. Remote sensing dominates the longest-chain group, and some single-call tools encapsulate several operations. These factors limit generalization from chain length to intrinsic task difficulty.

Name-based process metrics favor agreement with the reference implementation. They may penalize a valid alternative tool or order. PAS scores arguments only on steps aligned by tool name. Final-answer scores address the returned value and can miss missing artifacts when the reported scalar or choice happens to agree. Complete assessment requires evaluating the requested outputs and their provenance.

F.4 Experimental Attribution and Reuse

The reported configuration studies contain incomplete interventions and heterogeneous result provenance. Aggregate differences

cannot isolate all proposed module contributions, and no repeated-run uncertainty or matched efficiency analysis is supplied. External evaluation uses one model judge and requires verification of paired task identity and settings.

Candidate Skill representation provides a way to retain a verified composition. The present experiments do not demonstrate autonomous Skill registration, improved future-task coverage, or reduced planning cost. Evaluating reuse requires separating construction tasks from later tasks and checking that applicability and scientific assumptions remain valid under new inputs.